

Week 12: Strings and File Handling

1.Strings in C

In programming, data storage often involves arrays, which are collections of elements of the same type. Just as integers can be stored in an array of integers, [an array of characters is commonly known as strings](#). Strings play a vital role in programming as they allow the manipulation of textual data such as words and sentences.

In many programming languages, strings are handled as arrays of characters, though this implementation detail may be abstracted from the programmer. This means that operations on strings often involve treating them as sequences of individual characters stored in contiguous memory locations.

A crucial concept in string handling is the notion of a string constant, essentially a one-dimensional array of characters terminated by a special character known as the null character, represented as '\0'. This null character is crucial to determine where the string ends. For instance, consider the string constant:

```
char name[ ] = { 'M', 'O', 'D', 'I', 'J', 'I', '\0' };
```

In this example, the null character '\0' is appended at the end of the array to signify the termination of the string. It's important to note that the null character is distinct from the character '0'. The null character has an ASCII value of 0, while the character '0' has an ASCII value of 48.

C concedes the fact that you would use strings very often and hence provides a shortcut for initializing strings. For example, the string used above can also be initialized as,

```
char name[ ] = "MODIJI" ;
```

[Note that, in this declaration '\0' is not necessary. C inserts the null character automatically.](#)

1.1 Printing a string

Below is a snippet of code that prints a string

```
#include <stdio.h>

int main() {
    char name[] = "Klinsman";
    int i = 0;

    while (i <= 7) {
        printf("%c", name[i]);
        i++;
    }

    return 0;
}
```

No big deal. We have initialized a character array and then printed out the elements of this array within a while loop. Can we write the while loop without using the final value 7? We can; because we know that each character array always ends with a '\0'. The following program illustrates this.

```
#include <stdio.h>

int main() {
    char name[] = "Klinsman";
    int i = 0;

    while (name[i] != '\0') {
        printf("%c", name[i]);
        i++;
    }

    return 0;
}
```

There is another simple way of printing a string. However, note that this technique is not very useful when you want to manipulate the characters of the string while printing.

```
#include <stdio.h>

int main() {
    // Define a string variable
    char name[] = "Hello, world!";

    // Print the string
    printf("String: %s\n", name);

    return 0;
}
```

1.2 Input a string using scanf()

```
#include <stdio.h>

int main() {
    char name[25];

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Hello %s!", name);

    return 0;
}
```

In this code:

- We declare a character array name with a size of 25 to store the input string.
- We prompt the user to enter their name using printf.
- We use scanf with the %s format specifier to read the string input from the user and store it in the name array.
- Finally, we print a greeting message using printf, where %s is replaced with the entered name.

Note: When the string is an user input and not a predefined one, you have to declare an array with a size specified.

While entering the string using scanf() we must be cautious about two things:

(a) The length of the string should not exceed the dimension of the character array. This is because the C compiler doesn't perform bounds checking on character arrays. Hence, if you carelessly exceed the bounds there is always a danger of overwriting something important, and in that event, you would have nobody to blame but yourselves.

(b) scanf() is not capable of receiving multi-word strings. Therefore names such as 'Rinku Singh' would be unacceptable. We can get over this limitation using gets() function.

1.3 Input a string using gets()

```
#include <stdio.h>

int main() {
    char name[25];

    printf("Enter your full name: ");
    gets(name);

    puts("Hello!");
    puts(name);

    return 0;
}
```

And here is the output...

Enter your full name:

Hello!

Rinku Singh

The program is self-explanatory except for the fact that puts() can display only one string at a time (hence the use of two puts() in the program above). Also, on displaying a string, unlike printf(), puts() places the cursor on the next line.

1.4 Standard string manipulation functions in C

To use these functions, you need to include <string.h> header file.

Function	Description
strlen()	It returns the string's length.
strnlen()	It returns the specified value if the value specified is less than the string length, otherwise the string length.
strcmp()	It compares two strings and returns 0 if the strings are the same.
strncmp()	It compares two strings only to n characters.
strcat()	It concatenates two strings and returns the concatenated string.
strncat()	It concatenates n characters of one string to another string.
strcpy()	It copies one string into another.
strncpy()	It copies the first n characters of one string into another.
strchr()	It finds out the first occurrence of a given character in a string.
strrchr()	It finds out the last occurrence of a given character in a string.
strstr()	It finds out the first occurrence of a given string in a string.

strnstr()	It finds out the first occurrence of a given string in a string where the search is limited to n characters.
strcasecmp()	It compares two strings without sensitivity to the case.
strncasecmp()	It compares n characters of one string to another without sensitivity to the case.

Let's try out some of these functions:

strlen()

This function counts the number of characters present in a string. Its usage is illustrated in the following program.



```
#include <stdio.h>
#include <string.h>

int main() {

    char string1[20] = "Welcome to CS110";
    printf("Length of string string1: %ld", strlen(string1));
    return 0;
}
```

Output:

Length of string1: 16

strcmp()

This is a function which compares two strings to find out whether they are same or different. The two strings are compared character by character until there is a mismatch or end of one of the strings is reached, whichever occurs first. If the two strings are identical, strcmp() returns a value zero. If they're not, it returns the numeric difference between the ASCII values of the first non-matching pairs of characters. Here is a program which puts strcmp() in action.

```

#include <stdio.h>
#include <string.h>

int main() {

    char s1[20] = "Manchester"; // string1
    char s2[20] = "Manchester United"; // string2
    // comparing both the strings
    if (strcmp(s1, s2) == 0) {
        printf("string 1 and string 2 are equal");
    } else {
        printf("string 1 and 2 are different");
    }
}

```

Picyard

Output:

String 1 and 2 are different

strcpy()

This function concatenates the source string at the end of the target string. For example, “Bombay” and “Nagpur” on concatenation would result into a string “BombayNagpur”. Here is an example of strcat() at work.

```

#include <stdio.h>
#include <string.h>

int main(){
    char source[] = "Folks!";
    char target[30] = "Hello";

    strcat(target, source);

    printf("\nsource string = %s", source);
    printf("\ntarget string = %s", target);

    return 0;
}

```

Picyard

Output:

source string = Folks

Target string = Hello Folks

2. 2D Strings:

2.1 Introduction:

Two-dimensional arrays in C provide a convenient way to represent and manipulate tabular data, such as matrices, tables, and grids. When dealing with strings, a two-dimensional array can be used to store an array of strings, where each row represents a string. 1. Declaration and Initialization:

In C, a two-dimensional array is essentially an array of arrays. To declare and initialize a two-dimensional array of strings, follow this syntax:

```
#include <stdio.h>

#define ROWS 3
#define COLS 20

int main() {
    char strings[ROWS][COLS] = {
        "First string",
        "Second string",
        "Third string"
    };

    // Accessing and printing the strings
    for (int i = 0; i < ROWS; i++) {
        printf("%s\n", strings[i]);
    }

    return 0;
}
```


2.2 Accessing Elements:

You can access individual characters within a string in a two-dimensional array using array indices. For example:

```
char strings[2][20] = {
    "Hello",
    "World"
};

char first_char = strings[0][0]; // Accessing 'H'
char second_char = strings[1][1]; // Accessing 'o'
```

Picyard

2.3 Input/Output Operations:

Reading and printing two-dimensional strings is similar to reading and printing one-dimensional strings. Here's how you can do it:

```
#include <stdio.h>

#define ROWS 2
#define COLS 20

int main() {
    char strings[ROWS][COLS];

    // Input strings
    for (int i = 0; i < ROWS; i++) {
        printf("Enter string %d: ", i + 1);
        scanf("%s", strings[i]);
    }

    // Output strings
    printf("Entered strings:\n");
    for (int i = 0; i < ROWS; i++) {
        printf("%s\n", strings[i]);
    }

    return 0;
}
```

Picyard

2.4 Manipulating Strings:

You can manipulate two-dimensional strings using standard string manipulation functions like `strcpy`, `strcat`, `strlen`, etc. For example:



```
#include <stdio.h>
#include <string.h>

#define ROWS 2
#define COLS 20

int main() {
    char strings[ROWS][COLS] = {
        "Hello",
        "World"
    };

    // Concatenate two strings
    strcat(strings[0], " ");
    strcat(strings[0], strings[1]);

    printf("Concatenated string: %s\n", strings[0]);

    return 0;
}
```

Picyard

3. File Handling in C

File Operations encompass a variety of tasks that can be performed on files. These include:

- (a) Creation of a new file
- (b) Opening an existing file
- (c) Reading from a file
- (d) Writing to a file
- (e) Closing a file

Now, let's delve into a program designed to read a file and display its contents on the screen. We'll examine its functionality and dissect it line by line.

```

/* Display contents of a file on screen. */
#include "stdio.h"

int main() {
    FILE *fp;
    char ch;

    // Open the file "PR1.C" for reading
    fp = fopen("PR1.C", "r");

    // Read characters from the file and display them
    while (1) {
        ch = fgetc(fp);
        if (ch == EOF)
            break;
        printf("%c", ch);
    }

    // Close the file
    fclose(fp);

    return 0;
}

```

Picyard

Upon execution, this program reads and displays the contents of the file named "PR1.C" on the screen. Now, let's understand how it achieves this functionality.

3.1 Opening a File:

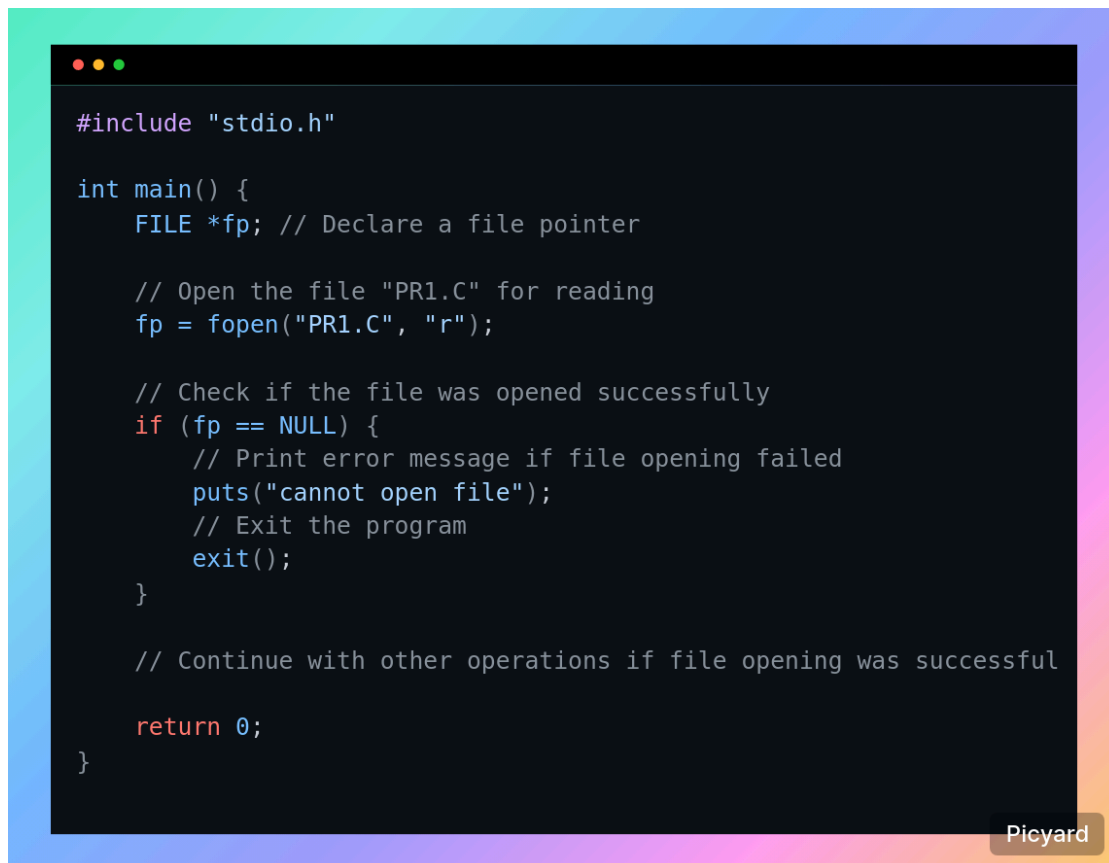
Before performing any file operations, it's necessary to open the file. In this program, the `fopen()` function is used to open the file "PR1.C" in read mode ("r"). The read mode indicates that the program intends to read the contents of the file.

Note: The mode argument ("r") is provided as a string enclosed in double quotes, indicating it's a string and not a character. This is important to remember while specifying file modes.

Now, let's proceed to understanding other aspects of this program.

There is a possibility that when we try to open a file using the function `fopen()`, the file may not be opened. While opening the file in "r" mode, this may happen because the file being opened may not be present on the disk at all. And you obviously cannot read a file that doesn't exist. Similarly, while opening the file for writing, `fopen()` may fail due to a number of reasons, like,

disk space may be insufficient to open a new file, or the disk may be write protected or the disk is damaged and so on. Crux of the matter is that it is important for any program that accesses disk files to check whether a file has been opened successfully before trying to read or write to the file. If the file opening fails due to any of the several reasons mentioned above, the `fopen( )` function returns a value `NULL` (defined in "stdio.h" as `#define NULL 0`). Here is how this can be handled in a program...



```
#include "stdio.h"

int main() {
    FILE *fp; // Declare a file pointer

    // Open the file "PR1.C" for reading
    fp = fopen("PR1.C", "r");

    // Check if the file was opened successfully
    if (fp == NULL) {
        // Print error message if file opening failed
        puts("cannot open file");
        // Exit the program
        exit();
    }

    // Continue with other operations if file opening was successful

    return 0;
}
```

Picyard

3.2 Closing a File:

When we have finished reading from the file, we need to close it. This is done using the function `fclose( )` through the statement, `fclose ( fp )`; Once we close the file we can no longer read from it using `getc( )` unless we reopen the file. Note that to close the file we don't use the filename but the file pointer `fp`. On closing the file the buffer associated with the file is removed from memory. In this program we have opened the file for reading. Suppose we open a file with an intention to write characters into it.

Sample Program to count chars, spaces, tabs and newlines in a file

```

/* Count chars, spaces, tabs and newlines in a file */
#include "stdio.h"

int main() {
    FILE *fp;
    char ch;
    int nol = 0, not = 0, nob = 0, noc = 0;

    // Open the file "PR1.C" for reading
    fp = fopen("PR1.C", "r");

    // Check if the file was opened successfully
    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1; // Return error code
    }

    // Read characters from the file and count characters, spaces, tabs, and newlines
    while (1) {
        ch = fgetc(fp);
        if (ch == EOF)
            break;
        noc++; // Count characters
        if (ch == ' ')
            nob++; // Count spaces
        if (ch == '\n')
            nol++; // Count newlines
        if (ch == '\t')
            not++; // Count tabs
    }

    // Close the file
    fclose(fp);

    // Print the counts
    printf("\nNumber of characters = %d", noc);
    printf("\nNumber of blanks = %d", nob);
    printf("\nNumber of tabs = %d", not);
    printf("\nNumber of lines = %d", nol);

    return 0;
}

```

Picyard

3.3 Writing to a File:

The `fputc()` function is used to write characters to a file. Which file? The file signified by the file pointer `ft`. The writing process continues till all characters from the source file have been written to the target file, following which the while loop terminates.

3.4 File handling Modes:

Mode	Description
'r'	Read from a file. Searches file, If the file is opened successfully fopen() loads it into memory and sets up a pointer which points to the first character in it. If the file cannot be opened fopen() returns NULL.
'w'	Write to a file. Searches file, If the file exists, its contents are overwritten. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file.
'a'	Appends at the end of a file. Searches file, If the file is opened successfully fopen() loads it into memory and sets up a pointer that points to the last character in it. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file.
'r+'	Reading existing contents, writing new contents, modifying existing contents of the file. Searches file, If is opened successfully fopen() loads it into memory and sets up a pointer which points to the first character in it. Returns NULL, if unable to open the file.
'w+'	Writing new contents, reading them back and modifying existing contents of the file. Searches file, If the file exists, its contents are overwritten. If the file doesn't exist a new file is created. Returns NULL, if unable to open file.
'a+'	Reading existing contents, appending new contents to end of file. Cannot modify existing contents. Searches file. If the file is opened successfully fopen() loads it into memory and sets up a pointer which points to the first character in it. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file.

3.5 String IO in Files:

For many purposes, character I/O is just what is needed. However, in some situations the usage of functions that read or write entire strings might turn out to be more efficient. Reading or writing strings of characters from and to files is as easy as reading and writing individual characters.

3.5.1 Here is a program that writes strings to a file using the function `fputs( )`.

```
/* Receives strings from keyboard and writes them to file */
#include "stdio.h"

int main() {
    FILE *fp;
    char s[80];

    // Open the file "POEM.TXT" for writing
    fp = fopen("POEM.TXT", "w");

    // Check if the file was opened successfully
    if (fp == NULL) {
        puts("Cannot open file");
        exit(1); // Exit the program with an error code
    }

    // Prompt the user to enter text
    printf("\nEnter a few lines of text:\n");

    // Loop to receive strings from the keyboard and write them to the file
    while (strlen(gets(s)) > 0) {
        // Write the string to the file
        fputs(s, fp);
        // Write a newline character to the file
        fputs("\n", fp);
    }

    // Close the file
    fclose(fp);

    return 0;
}
```

Explanation:

- The fopen() function is used to open the file "POEM.TXT" for writing ("w" mode).
- The code checks if the file was opened successfully. If not, it prints an error message and exits the program with an error code.
- The user is prompted to enter a few lines of text.
- Inside the while loop, the gets() function is used to read a line of text from the keyboard into the array s. The loop continues as long as the length of the input string (obtained using strlen()) is greater than 0.
- Inside the loop, the input string is written to the file using the fputs() function, followed by a newline character to ensure each string is written on a separate line.
- After writing all input strings to the file, the file is closed using the fclose() function.
- Finally, the program returns 0 to indicate successful execution.

3.5.1 Here is a program that reads strings from a file using the function fgets().

```
/* Reads strings from the file and displays them on screen */
#include "stdio.h"

int main() {
    FILE *fp;
    char s[80];

    // Open the file "POEM.TXT" for writing
    fp = fopen("POEM.TXT", "w");

    // Check if the file was opened successfully
    if (fp == NULL) {
        puts("Cannot open file");
        exit(1); // Exit the program with an error code
    }

    // Prompt the user to enter text
    printf("\nEnter a few lines of text:\n");

    // Loop to receive strings from the keyboard and write them to the file
    while (strlen(gets(s)) > 0) {
        // Write the string to the file
        fputs(s, fp);
        // Write a newline character to the file
        fputs("\n", fp);
    }

    // Close the file
    fclose(fp);

    return 0;
}
```


Explanation:

- The `fopen()` function is used to open the file "POEM.TXT" for reading ("r" mode).
- The code checks if the file was opened successfully. If not, it prints an error message and exits the program with an error code.
- Inside the while loop, the `fgets()` function is used to read a line of text (up to 79 characters) from the file into the array `s`. The loop continues until `fgets()` returns `NULL`, indicating the end of the file.
- Inside the loop, each string read from the file is displayed on the screen using the `printf()` function.
- After reading all strings from the file, the file is closed using the `fclose()` function.
- Finally, the program returns 0 to indicate successful execution.